Problem 1e. Further Model Size Reduction:

1. Comparison of previous models

Model	Size (KB)	Test Accuracy
Base model (float32)	14.08	100.00%
Pruned, sparse-encoded	8.08	98.15%
Dynamic-range PTQ	8.55	100.00%
Float16 PTQ	8.95	100.00%
KD student (float32)	6.12	100.00%

The smallest technique was the KD student, which gave 6.12 KB. It also had the best performance with 100% accuracy, although they all had very similar results.

2. Strategy Proposal

The strategy I chose to do was use all 3 compression techniques to shrink the model as small as possible. I first used knowledge distillation, because that provided the smallest model, then I applied pruning to remove the redundant connections, and then I applied a full int8 PTQ to shrink the weights. Because these techniques all work independently, applying them all should be effective.

3. Implementation

I first cloned the KD student model to get a fresh copy that we can prune and quantize. Then, I wrapped its layers using the same pruning schedule from part 1c, and trained for 10 epochs. Then, I stripped the wrappers and converted to TensorFlowLite with a full int8 quantization using the same quantize_and_evaluate() helper from part 1b.

4. Evaluation

The final model yielded a size of only 4.77 KB, which is 66% smaller than the original model and 22% smaller than the KD student. The accuracy was 98.15% instead of 100% like the KD student was.

5. Justification

I successfully reduced the size further than any individual technique, with only a small accuracy cost. The biggest factor in reducing the model size was the knowledge distillation, then the quantization, then the pruning.

Problem 2:

1.

Classification: classify a person saying yes or no.
Regression: Predicting the temperature using a light and humidity sensor
Transfer Learning (images): Sort photos into apple vs banana using pictures of each
Transfer Learning (kws): wake a device when you say alexa
Object detection (images): count the number of people in a room
Anomaly Detection (gmm): flag when a fans vibration is not normal
Anomaly Detection (k-means): detect that a water pump is jammed
Visual Anomaly Detection (FOMO-AD): spot scratched phone after only ever seeing a clean one
Anomaly Detection: Same as previous, using your custom ml

2.

- Dense (fully connected): every neuron connects to every neuron in the previous layer. This is the general purpose ML layer. It does the final mapping of features to classes and can sometimes be the whole network.
- 1D convolution: learns features that take spatial information into account for a single dimension. Filter slides across one axis and detects patterns, then shrinks the feature map to detect where in the window the pattern occurs. Use for MFCC.
- 2D convolution: same as 1d but with 2 dimensions. Can be used for images and for time/frequency representations like spectrograms
- Reshape: turns one-dimensional into multi-dimensional data necessary for convolution layers to identify patterns,
- Flatten: collapse multi-dimensional data into a single dimension. Must be used after a convolution before the output because dense layers take a flat vector.
- Dropout: randomly cuts part of the network during training to reduce overfitting. Useful when training performance is much better than validation performance.

3.

The model compression options are the EON compiler instead of tensorflow lite, and it can use int8 quantization instead of unoptimized float32.

4.

Synthetic data can be used with images using either GPT-Image-2 Synthetic Image Generator, or Flux-Pro Synthetic Image Generator. For speech/audio, Whisper Synthetic Voice Generator is available. Synthetic data is important because it can be cheaper than manual data collection and can be used to cover edge cases. It can also be used to fill gaps in a real dataset that you have. However, if it is trained on only synthetic data, it might not perform as well on real data.